

“Multi-Agent AI Execution and Resource Management System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

S.DIVYA SAI	2520039622
AYUSHA DAS	2520039624
M.V.S.R HITESH	2520039621
CHANDRA PRAKASH VERMA	2500031353

Under the Esteemed Guidance of

DR. HEMA

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “Multi-Agent AI Execution and Resource Management System” is a record of Bonafide work of S. DIVYA SAI - 2520039622, AYUSHA DAS - 2520039624, M.V.S.R HITESH - 2520039621, and CHANDRAPRAKASH VEMA - 2500031353 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have been prepared for the Operating Systems and Systems Programming project.

The report presents the project objective, system architecture, Operating System concepts, implementation, execution, and results of the Multi-Agent AI Execution and Resource Management System.

S.DIVYA SAI	2520039622
AYUSHA DAS	2520039624
M.V.S.R HITESH	2520039621
CHANDRAPRAKASH VERMA	2500031353

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled “Multi-Agent AI Execution and Resource Management System” is a bonafide work done and submitted by S. DIVYA SAI, AYUSHA DAS, M.V.S.R HITESH and CHANDRAPRAKASH VEMA, in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2026–27, Term-I.

The project demonstrates process management, inter-process communication, file management, synchronization, resource monitoring, signals, logging and backup mechanisms in a Linux-based C application.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of this project.

Our greatest appreciation goes to our project guide, Dr. Hema, Department of Computer Science and Engineering, for her guidance, support and encouragement throughout the development of the project.

We express our gratitude to the Department of Computer Science and Engineering and K L (Deemed to be) University for providing the academic environment and facilities required to complete this Operating Systems and Systems Programming project.

We thank all the members of the teaching and non-teaching staff, our team members, friends and family for their cooperation and support during the work.

S.DIVYA SAI 2520039622

AYUSHA DAS 2520039624

M.V.S.R HITESH 2520039621

CHANDRA PRAKASH VERMA 2500031353

Abstract

The Multi-Agent AI Execution and Resource Management System is a Linux-based C application designed to provide a secure and controlled environment for running multiple AI agents simultaneously. In an organization, uncontrolled agents may cause unauthorized file access, excessive CPU and memory usage, file conflicts, and system instability. The proposed system addresses these problems by applying important Operating System and System Programming concepts.

Each AI agent is created as a separate process, allowing independent execution and monitoring. Named Pipes (FIFOs) are used for Inter-Process Communication to exchange structured task and execution information between agents. A restricted workspace is provided to control file operations, and file-access validation prevents unauthorized access. The system monitors CPU and memory consumption and uses signals to control or terminate agents that exceed predefined resource limits. Synchronization mechanisms are implemented to prevent conflicts when multiple agents access shared files.

The system also records agent activities and execution results automatically. Backup copies of modified workspace files are maintained to support data recovery and prevent loss of important information. The expected outcome is a secure, reliable, and efficient multi-agent execution environment that manages system resources effectively, protects files, and demonstrates practical Linux-based Operating System and System Programming concepts.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1
		1.1 Background of the Project	1
		1.2 Problem Statement	1
		1.3 Scope of the Project	1
		1.4 Objectives of the Project	1
		1.5 Importance of the Application	1
		1.6 Target Users / Audience	1
2	OS Concepts and Background Study		2
		2.1 Operating System Concepts Related to the Project	2
		2.2 Existing Methods / Algorithms	2
		2.3 Proposed Approach	2
		2.4 Advantages of the Proposed Approach	2
3	System Requirements		3
		3.1 Hardware Requirements	3
		3.2 Software Requirements	3
		3.3 Operating System Environment – Ubuntu / WSL	3
		3.4 Development Tools and Compilers	3
4	Technology and Tools Used		4
		4.1 C / C++ Programming	4
		4.2 GCC Compiler	4
		4.3 Linux System Calls	4
		4.4 POSIX Threads / Synchronization	4
		4.5 Linux Commands and Utilities	4
		4.6 Other Tools / Libraries Used	4
5	System Architecture and Design		5–6
		5.1 Overall System Architecture	5
		5.2 System Workflow	5
		5.3 Flowchart / Data Flow	5
		5.4 Module Description	5

		5.5 Input and Output Design	6
6	Implementation		7–8
		6.1 Module-wise Implementation	7
		6.2 Algorithm / Pseudocode	7
		6.3 Core C / C++ Implementation	7
		6.4 Linux System Calls / APIs Used	7
		6.5 Process / Thread Implementation	8
		6.6 IPC / Synchronization Implementation	8
		6.7 Memory Management Implementation	8
		6.8 Optional User Interface Implementation	8
7	Execution and Demonstration		9–10
		7.1 Compilation in Ubuntu	9
		7.2 Execution Commands	9
		7.3 Sample Inputs and Outputs	9
		7.4 Screenshots of Execution	9
		7.5 Demonstration of OS Concepts	10
8	Conclusion		11
		8.1 Summary of the Project	11
		8.2 Objectives Achieved	11
		8.3 Results Achieved	11
		8.4 Skills and Learning Outcomes	11
9	References		12
		9.1 Books	12
		9.2 Research Papers	12
		9.3 Linux / GNU Documentation	12
		9.4 Online Resources	12

CHAPTER -1 INTRODUCTION

CHAPTER -1 INTRODUCTION

CHAPTER -2 SYSTEM REQUIREMENTS

CHAPTER -2 SYSTEM REQUIREMENTS

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

The Multi-Agent AI Execution and Resource Management System is a Linux-based Operating Systems and Systems Programming project that demonstrates how multiple independently managed agent processes can be executed in a controlled environment. The project applies Linux process management, inter-process communication, file management, synchronization, resource monitoring, signals, logging and backup mechanisms within a single application.

The system follows a manager-agent architecture. A Main Controller starts and coordinates four agent processes: Code Generator, Code Review Agent, Execution Agent and Testing Agent. These agents perform separate stages of a controlled program-generation and execution pipeline and communicate through Named Pipes (FIFOs).

1.2 Problem Statement

Organizations need to run multiple AI agents simultaneously in a secure and controlled environment. Without proper management, agents may access unauthorized files, consume excessive CPU or memory, and cause conflicts when accessing shared files. The project addresses these issues using Operating System and System Programming concepts such as process management, IPC, file access control, resource monitoring, signals and synchronization to ensure safe and efficient agent execution.

1.3 Scope of the Project

The scope of the project is limited to a local Linux environment in which multiple independent agent processes are created, supervised and coordinated. The implementation covers controlled workspace access, FIFO-based communication, file synchronization, backup, logging, resource limits, signals, compilation and execution of generated C programs, testing and run-history recording.

The project does not include an external AI/LLM service, machine-learning model, cloud service, database, web application, REST API or authentication system. The term AI agent refers to an independently managed process performing a controlled task in the project pipeline.

1.4 Objectives of the Project

- To create and manage multiple AI agents as separate processes in a Linux environment.
- To enable communication between agents using Named Pipes (FIFOs).
- To control file access and monitor CPU and memory usage of each agent.
- To ensure safe execution through signals, synchronization, activity logging and file backups.

1.5 Importance of the Application

The application provides a practical demonstration of core operating-system mechanisms. It shows how process creation, process execution, process waiting, IPC, file descriptors, file locking, signals and resource monitoring can be combined to build a controlled concurrent system. It also demonstrates why file protection, synchronization and recovery mechanisms are necessary when multiple processes operate on shared resources.

1.6 Target Users / Audience

The primary audience is students and developers studying Operating Systems and Systems Programming on Linux. The project can also be used as an academic demonstration of process management, IPC, file management, synchronization, resource monitoring and system-level programming.

CHAPTER -2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The project applies several Operating System concepts directly in its implementation. Process management is used to create and supervise independent agent processes. Named Pipes provide inter-process communication. File descriptors and Linux system calls provide low-level file management. Signals provide process control and termination, while file locking provides synchronization for protected shared-file operations.

The major concepts used are process creation using `fork()`, process execution using `exec*()`, process synchronization using `waitpid()`, file management using `open()`, `read()`, `write()` and `close()`, FIFO-based IPC, signal handling, file locking using `flock()`, restricted workspace validation and resource monitoring.

2.2 Existing Methods / Algorithms

A basic multi-process program can create child processes and allow them to execute independently, but without coordination it may permit uncontrolled resource usage and conflicting access to shared files. Traditional file handling also requires explicit validation and synchronization when more than one process can access a resource.

The project improves this basic approach by adding a restricted workspace, canonical path validation, FIFO communication, exclusive file locking, backups, execution timeouts, logging and process supervision. These mechanisms are integrated into a single pipeline rather than being demonstrated as unrelated examples.

2.3 Proposed Approach

The proposed approach creates separate processes for the Code Generator, Code Review Agent, Execution Agent and Testing Agent. The stages communicate through named FIFOs. Before execution, the requested source file is validated against the restricted workspace. The protected workspace operation obtains an exclusive `flock()` lock and creates a backup. The source is then compiled and the resulting program is executed under a timeout. The Testing Agent receives the execution result and the Main Controller records the run history.

2.4 Advantages of the Proposed Approach

- Process-level separation between agent stages.
- Structured IPC using Named Pipes.
- Restricted workspace access to reduce unauthorized file operations.
- Exclusive file synchronization using `flock()`.
- Backup before protected workspace operations.
- Resource limits and execution timeout mechanisms.
- Activity logging and persistent run history.
- Failure handling for invalid files, FIFO timeouts, compilation errors and program timeouts.

CHAPTER -3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The project is designed for a standard computer capable of running a Linux environment and compiling C programs. A practical development configuration is a dual-core or better processor, at least 4 GB RAM and sufficient free storage for the source files, generated output, logs and backup files.

3.2 Software Requirements

- Linux / Ubuntu operating system or an equivalent POSIX-compatible Linux environment.
- GCC compiler for compiling the project and generated C programs.
- C programming language and standard Linux/POSIX libraries.
- Terminal / command-line environment for compilation and execution.
- Visual Studio Code or another source-code editor, if required.
- Git/GitHub for team source-code management, if used by the team.

3.3 Operating System Environment – Ubuntu / WSL

The implementation targets Linux because it directly uses Linux/POSIX mechanisms such as fork(), waitpid(), mkfifo(), flock(), access(), realpath(), kill(), signals and file descriptors. Ubuntu or Ubuntu under WSL provides the required Linux user-space environment for compiling and executing the project.

3.4 Development Tools and Compilers

GCC is used to compile the C source code. A terminal is used to run compilation commands and execute the resulting binary. Visual Studio Code may be used for source editing. The final project executable is intended to run from the project directory so that its FIFO, workspace, backup and log paths remain consistent.

CHAPTER -4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The project is implemented in C. C is appropriate for the project because it provides direct access to Linux system calls, file descriptors, process APIs, signal handling and POSIX facilities. The implementation uses standard C library functionality together with Linux/POSIX headers.

4.2 GCC Compiler

GCC is used to compile the project source and also to compile generated C programs during the Execution Agent stage. The compiler child uses `execlp()` to execute gcc with the selected source file and output binary.

4.3 Linux System Calls

The implementation uses `open()`, `read()`, `write()`, `close()`, `fork()`, `waitpid()`, `mkfifo()`, `access()`, `realpath()`, `kill()` and related Linux/POSIX APIs. These calls provide the low-level mechanisms required for process, file, communication and control operations.

4.4 POSIX Threads / Synchronization

The project uses POSIX/Linux synchronization mechanisms and file locking. The protected workspace operation uses `flock()` with `LOCK_EX` for an exclusive lock and `LOCK_UN` for release. Where the implementation uses thread-related monitoring or shared state, POSIX thread facilities are used to coordinate those activities.

4.5 Linux Commands and Utilities

Typical Linux commands used during development and demonstration include gcc for compilation, `./program` for execution, `ls` for checking generated files, `cat` for viewing text files, `mkdir` for creating directories and `pwd` for checking the working directory.

4.6 Other Tools / Libraries Used

The implementation uses Linux/POSIX headers including `unistd.h`, `sys/types.h`, `sys/wait.h`, `sys/stat.h`, `fcntl.h`, `signal.h`, `sys/file.h`, `errno.h`, `limits.h`, `dirent.h` and `time.h`. These provide the APIs required by the actual source code.

CHAPTER -5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system follows a Main Controller and Agent Process architecture. The Main Controller prepares the environment, selects the program, creates the four agent processes and waits for their completion. The agents communicate through three FIFO channels and use a common restricted workspace.

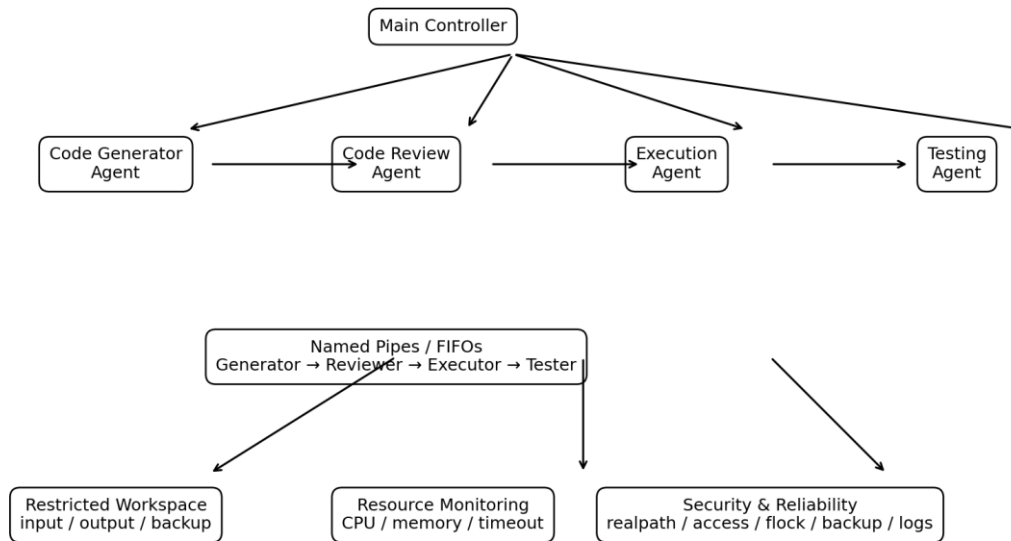


Figure 5.1 Overall Architecture of the Multi-Agent AI Execution and Resource Management System

5.2 System Workflow

The workflow starts when the user selects a catalog program or a custom program. The Main Controller prepares the workspace and creates the four agent processes. The Code Generator prepares the source, the Code Review Agent performs static checks, the Execution Agent validates and synchronizes the file before compilation and execution, and the Testing Agent evaluates the result. The Main Controller then records the run history and displays the pipeline summary.



Resource monitoring, signals, activity logging and run history operate around the pipeline.

Figure 5.2 System Workflow

5.3 Flowchart / Data Flow

The data flow is Generator → Reviewer → Executor → Tester. A TaskMessage structure carries agent name, task, file name, status, result and expected output information between the stages. The FIFO endpoints provide the communication channels while the workspace stores source, output, backup, log and result files.

5.4 Module Description

- Main Controller – prepares the environment, starts the pipeline, monitors execution and records results.
- Code Generator Agent – generates or selects the C source program.

- Code Review Agent – receives the generated source and performs static review checks.
- Execution Agent – validates file access, synchronizes and backs up the source, compiles it and executes the resulting program.
- Testing Agent – receives the execution result and produces the final test verdict.
- File Guard – validates that requested files remain inside the restricted workspace.
- Backup Module – copies workspace files to the backup directory before protected operations.
- Logger – records activity and execution information.
- Resource Monitor – observes configured CPU/memory limits and supports process control.

5.5 Input and Output Design

Input consists of a program selection or custom C source. Output includes terminal status messages, generated source files, compiled output, compiler diagnostics, activity logs, backup files, the last test result and persistent run history.

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

The implementation is organized around the Main Controller and four agent functions. The Main Controller prepares the runtime environment and starts the pipeline. Each agent performs one stage and communicates through a FIFO. File validation, synchronization, backup, compilation, execution, testing and logging are performed by the corresponding modules.

6.2 Algorithm / Pseudocode

Listing 6.1 — Overall Pipeline Pseudocode

```
START
Prepare FIFO, workspace and log directories
Load runtime configuration
Acquire single-instance lock
Select program
Create Code Generator process
Create Code Review process
Create Execution process
Create Testing process
Generator sends source information through FIFO
Reviewer checks generated source
Execution validates file and obtains exclusive lock
Create backup
Compile source
Execute compiled program with timeout
Testing Agent receives execution result
Main Controller waits for all agents
Record run history
Display pipeline summary
END
```

6.3 Core C / C++ Implementation

The actual implementation uses C functions corresponding to each OS concept. The following short examples show the key system-level operations rather than reproducing the complete source file.

Listing 6.2 — Agent Process Creation

```
pid_t generator_pid = fork();

if (generator_pid == 0)
{
    code_generator();
}
```

Listing 6.3 — File Management Operations

```
int fd = open(filename, O_RDONLY);
ssize_t bytes = read(fd, buffer, sizeof(buffer));
close(fd);
```

Listing 6.4 — File Synchronization

```
int fd = open(filename, O_RDWR);

if (flock(fd, LOCK_EX) == -1)
    return 0;

/* protected file operation */

flock(fd, LOCK_UN);
close(fd);
```

6.4 Linux System Calls / APIs Used

fork() creates child processes. execlp() replaces the compiler child with GCC and execl() replaces the execution child with the compiled program. waitpid() waits for child completion. open(), read(), write() and close() provide file management. mkfifo() creates Named Pipes. realpath() canonicalizes paths, access() checks file accessibility, flock() provides exclusive file locking, and kill() and signal-related APIs control processes.

6.5 Process / Thread Implementation

The Main Controller creates four independent agent processes using fork(). Child PIDs are stored so that the controller can monitor and wait for them. The Execution Agent also creates child processes for GCC compilation and execution of the generated program. waitpid() is used to collect child completion status.

6.6 IPC / Synchronization Implementation

Three Named Pipes are used in the pipeline: fifo/generator_to_reviewer, fifo/reviewer_to_executor and fifo/executor_to_tester. The implementation uses open(), read(), write() and close() on the FIFO descriptors. FIFO opening includes a timeout so a missing peer does not block the pipeline indefinitely. File synchronization uses

flock() with an exclusive lock.

6.7 Memory Management Implementation

The implementation primarily uses fixed-size buffers and structures for TaskMessage data, file copying and compiler diagnostics. Dynamic memory is not a central feature of the pipeline. File descriptors and temporary resources are explicitly closed after their operations.

6.8 Optional User Interface Implementation

The application provides a terminal-based interactive menu. The menu supports running the multi-agent system, running all catalog demonstrations, viewing the activity log, viewing backup files, viewing run history, resetting the environment and exiting. A command-line --run mode is also provided for non-interactive execution.

CHAPTER -7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

The project is compiled using GCC from the Linux terminal. The exact source filename should match the final file supplied by the team. A typical compilation form for the project is:

Listing 7.1 — Project Compilation Command

```
gcc -Wall -Wextra -std=gnu11 -O2 -o program program.c
```

7.2 Execution Commands

Listing 7.2 — Execution Commands

```
./program  
./a.out
```

The command `./a.out` starts the interactive Multi-Agent AI Execution System. The main menu allows the user to start the multi-agent pipeline, run demonstrations, view logs and backups, view run history, reset the environment and exit.

7.3 Sample Inputs and Outputs

A typical demonstration selects a catalog program such as Factorial Calculator. The system prepares the workspace, creates the four agent processes, transfers task information through FIFOs, validates the source, performs synchronization and backup, compiles the program, executes it and sends the result to the Testing Agent.

For the individual File Management module, the demonstrated sequence is file opening, data writing, file reading and file closing. The output confirms that each operation completed successfully.

7.4 Screenshots of Execution

The following screenshot shows the actual Multi-Agent AI Execution System running in the Linux terminal. It demonstrates compilation and execution of the project, Main Controller startup, the interactive main menu, workspace preparation and validation, and selection of a catalog program for the Code Generator.

```
chandra_prakash@LAPTOP-7ET89C9T:~$ gcc project6.c  
chandra_prakash@LAPTOP-7ET89C9T:~$ ./a.out
```

MULTI-AGENT AI EXECUTION SYSTEM

Main Controller started (PID 616)

MAIN MENU

1. Run Multi-Agent System
2. Run All Demos (all 5 catalog programs, back to back)
3. View Activity Log
4. View Backup Files
5. View Run History
6. Reset Environment
7. Exit

Enter your choice: 1

STARTING MULTI-AGENT SYSTEM

Preparing workspace, log, and IPC directories...
✓ Environment ready

Checking workspace access...
✓ Workspace access validated
 /home/chandra_prakash/workspace/input
✓ Workspace is accessible

SELECT A PROGRAM FOR THE CODE GENERATOR

1. Factorial calculator
2. Fibonacci sequence
3. Prime number check
4. Digit sum calculator
5. Unsafe demo (deliberately fails code review)
6. Custom program (paste your own C code)

Enter your choice [1-6]: 1

Figure 7.1 — Multi-Agent AI Execution System Running in Linux Terminal

7.5 Demonstration of OS Concepts

- Process Management: `fork()` creates independent agent processes and `waitpid()` collects their completion status.
- IPC: Named Pipes transfer structured TaskMessage information between agent processes.
- File Management: `open()`, `read()`, `write()` and `close()` manage workspace, backup and log files.
- File Security: `realpath()` and `access()` restrict file operations to the workspace.
- Synchronization: `flock()` provides an exclusive lock during protected file operations.
- Signals: SIGINT, SIGTERM, SIGALRM and SIGKILL are used for interruption, termination and timeout control.
- Resource Management: configured CPU/memory limits and execution timeouts provide resource-control mechanisms.
- Logging and Backup: activity logs, run history and backup files provide traceability and recovery support.

CHAPTER -8 CONCLUSION

8.1 Summary of the Project

The Multi-Agent AI Execution and Resource Management System demonstrates a controlled Linux environment for multiple independently managed agent processes. The system combines process management, FIFO-based IPC, restricted file access, synchronization, resource monitoring, signals, logging and backup into one execution pipeline.

8.2 Objectives Achieved

The project achieves the stated objectives by creating separate processes for agents, enabling FIFO communication, restricting workspace access, monitoring resource usage, applying synchronization and maintaining activity and backup information. The implementation also provides compilation, execution, testing and failure-handling mechanisms.

8.3 Results Achieved

The project produces a working Linux-based application in which the pipeline can generate or select a C program, review it, validate the associated file, synchronize and back up the workspace file, compile and execute the program, test the result and record the run history. The individual File Management module was also compiled and executed successfully, demonstrating open, write, read and close operations.

8.4 Skills and Learning Outcomes

The project provides practical understanding of Linux process management, system calls, file descriptors, IPC, synchronization, signals, resource control, error handling, GCC compilation and Linux terminal-based development. It also demonstrates how multiple OS concepts can be integrated into a single working system.

CHAPTER -9 REFERENCES

9.1 Books

- [1] Abraham Silberschatz, Peter B. Galvin and Greg Gagne, Operating System Concepts, Wiley.
- [2] W. Richard Stevens and Stephen A. Rago, Advanced Programming in the UNIX Environment, Addison-Wesley.

9.2 Research Papers

The project is primarily an Operating Systems and Systems Programming implementation. Any research-paper references used by the team for the final submission should be inserted here without claiming that they directly implement this project.

9.3 Linux / GNU Documentation

- [3] Linux/GNU documentation and manual pages for `fork()`, `exec()`, `waitpid()`, `open()`, `read()`, `write()`, `close()`, `mkfifo()`, `access()`, `realpath()`, `flock()`, `kill()` and signal handling.

9.4 Online Resources

- [4] POSIX/Linux system programming documentation consulted for process management, file management, IPC, synchronization and signal APIs.
- [5] Official project problem statement and submitted source code used as the primary project-specific references.